

Representation of Graph

Adjacency Matrix

u\v	1	2	3	4	5	6	7
1	0	1	1	1	0	0	0
2	0	0	0	1	1	0	0
3	0	0	0	0	0	1	0
4	0	0	1	0	0	1	1
5	0	0	0	1	0	0	1
6	0	0	0	0	0	0	0
7	0	0	0	0	0	1	0

Adjacency Matrix ไม่นิยมใช้ เพราะ เปลืองเนื้อที่ในหน่วยความจำ

49 ช่อง มีช่องที่เป็น 1 อยู่ 12 ช่อง

100 ช่อง มีช่องที่เป็น 1 อยู่ $= (12/49) \times 100 = 24.48 \%$

มีช่องที่เป็น 0 หรือไม่ได้ใช้งาน อยู่ 75.52%

What percentage of the memory space is used for entries with a value of 1, and what percentage is used for entries with a value of 0?

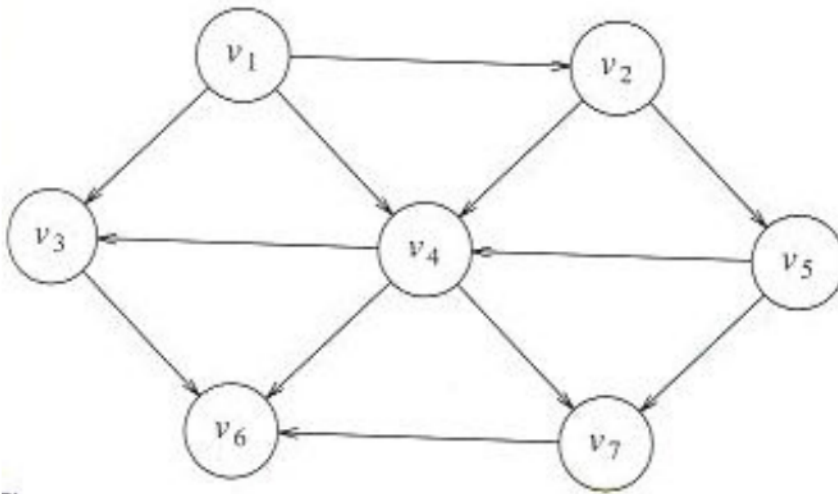
📌 คำตอบสรุปเปอร์เซ็นต์พื้นที่ในหน่วยความจำ (Official Answers):

- Entries with value of 1 (Used): $(12 / 49) \times 100 = 24.48 \%$
- Entries with value of 0 (Wasted): $(37 / 49) \times 100 = 75.52 \%$ (สูญเปล่าใน RAM)

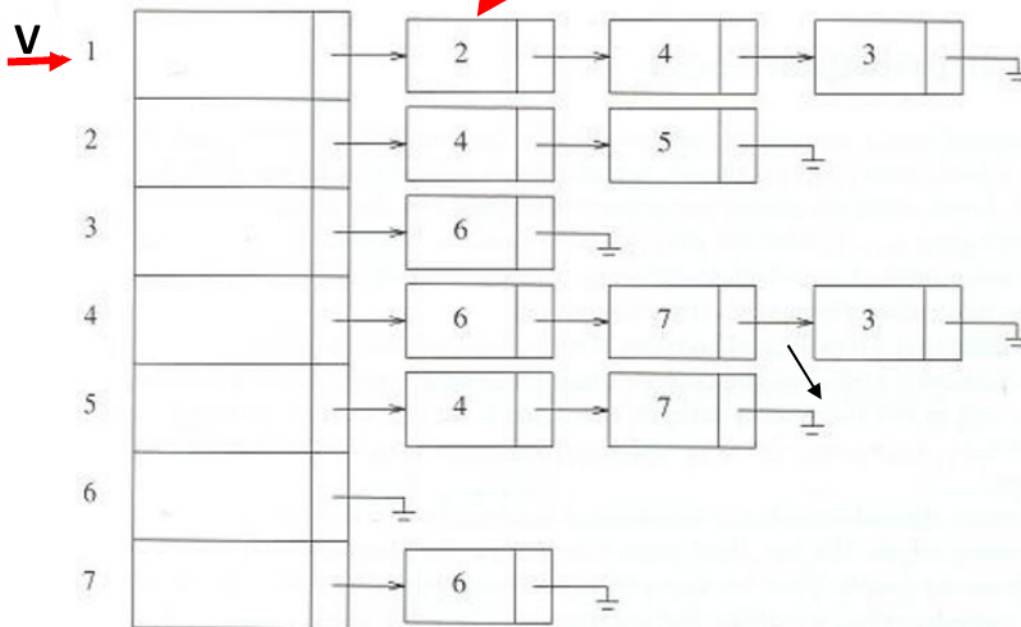
💡 **สาระสำคัญ: ทำไม Adjacency Matrix จึงไม่นิยมใช้ในทางปฏิบัติ?**

1. ความสูญเปล่ามหาศาล: ในโจทย์นี้เสียพื้นที่เก็บเลข 0 ไปเปล่าๆ ถึง 75.52% (37 จาก 49 ช่อง)
 2. โครงสร้าง Sparse Graph: ในโลกจริง เช่น Social Network หากมี 10,000 คน Matrix ต้องจองถึง 100 ล้านช่อง!
 3. เปรียบเทียบกับ Adjacency List: ใช้เนื้อที่เพียง $|V| + |E| = 7 + 12 = 19$ ช่อง (ประหยัด RAM กว่า 61.22%)
 4. ความซับซ้อนในการค้นหาเพื่อนบ้าน (Find Neighbors):
 - Matrix ใช้เวลา $O(|V|)$ = ต้องวนลูปตรวจสอบทั้งแถว 7 ช่อง แม้ว่าจะมีเส้นเชื่อมจริงแค่ 1-2 เส้น
 - Adjacency List ใช้เวลา $O(\deg(v))$ = ร้องตาม Pointer เท่าจำนวนเพื่อนบ้านจริงเท่านั้น เร็วกว่ามาก
- ✓ สรุป: นิยมแปลงกราฟเป็น Adjacency List ก่อนนำไปรัน Topological Sort หรือ Shortest Path เสมอ

Topological Sort



1. แปลงภาพกราฟ มาเป็น **adjacency list** ก่อน



💡 วิเคราะห์โครงสร้างหน่วยความจำ Adjacency List (Zero-Occlusion):

1. อาร์เรย์ Header Pointers (ขนาด 7 ช่อง, Index 1..7):

ทำหน้าที่เก็บพอยน์เตอร์ชี้ไปยังโหนดแรกของ Linked List ของแต่ละจุดยอดต้นทาง

2. โหนด Linked List แต่ละก้อนเก็บ 2 ส่วน: [Dest (ปลายทาง) | Next* (พอยน์เตอร์ชี้ก้อนถัดไป)]

3. สัญลักษณ์กราฟด์ (≡): แทนค่าพอยน์เตอร์ NULL บ่งชี้ว่า 'สิ้นสุดรายการเพื่อนบ้านของโหนดนี้'

4. รวมใช้พื้นที่ Memory ทั้งสิ้น: $|V| + |E| = 7 + 12 = 19$ ช่อง (ประหยัดหน่วยความจำกว่า Matrix 61.22%)

5. ลูกศรสีแดง v ชี้โหนด 1 และชี้โหนด 2: แสดงจุดเริ่มต้นการเข้าถึงเพื่อนบ้านตัวแรกของโหนดต้นทาง v

✓ สรุป: โครงสร้างนี้เป็นรากฐานสำคัญที่ส่งต่อไปยังตาราง Topological Sort (หน้า 3) และ Shortest Path (หน้า 4)

2. Indegree Array อะเรย์ของจำนวน edge ที่ชี้เข้าหา vertex

v	เริ่มต้น	รอบที่ 1	รอบที่ 2	รอบที่ 3	รอบที่ 4	รอบที่ 5	รอบที่ 6	q	
1	0	-	-	-	-	-	-	6	6: Enqueue(6)
2	1	0	-	-	-	-	-	5	3: Enqueue(3)
3	2	1	1	1	0	-	-	4	7: Enqueue(7)
4	3	2	1	0	-	-	-	3	4: Enqueue(4)
5	1	1	0	-	-	-	-	2	5: Enqueue(5)
6	3	3	3	3	2	1	0	1	2: Enqueue(2)
7	2	2	2	1	0	0	-	0	1: เริ่มต้น

สถานะพอยน์เตอร์เมื่อจบการทำงาน:

- Currentsize = 0 (คิวว่างเปล่าเมื่อจบ)
- Front = 0 (ดัชนีรอบครบกลับมาที่ 0)
- back = 6 (Enqueue คสว 7 จุดยอด)

Indegree array q.enqueue() (v) // นำโหนดที่ indegree=0 เข้าคิว

V= q.dequeue() // ดึงโหนดออกจากคิวมาประมวลผล

for each w adjacent to v W= เพื่อนบ้านของ v -> ลด indegree[w] ลง 1

จงเขียน Topological Sort ที่เป็นผลลัพธ์ จากกราฟดังกล่าว

After computing the topological sort, write down the topological ordering that is the solution for this graph.

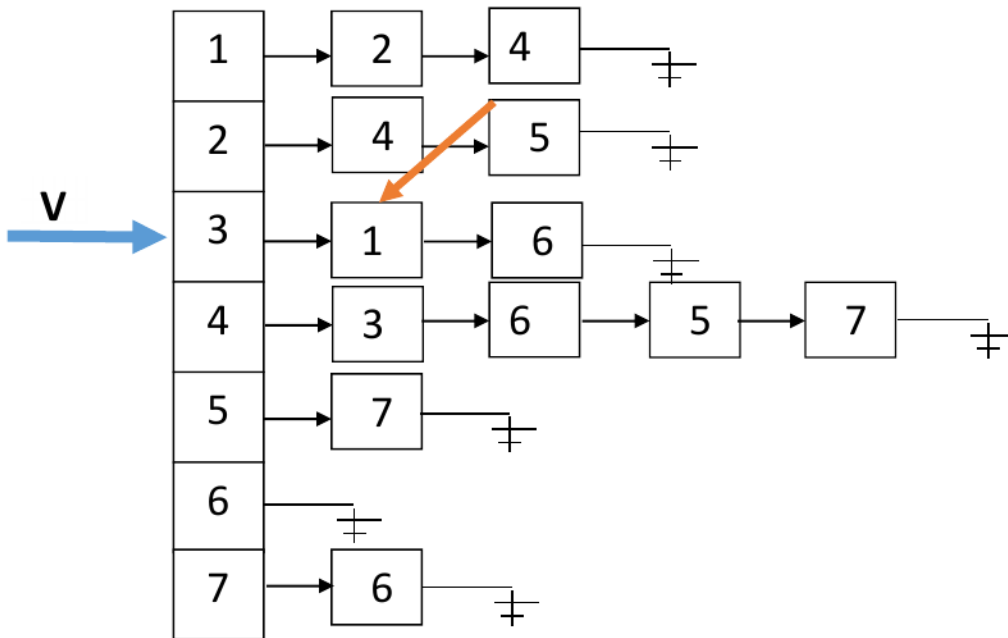
📌 คำตอบ Topological Ordering ที่ถูกต้องอย่างเป็นทางการ:

1, 2, 5, 4, 3, 7, 6 (หรือเขียน 1, 2, 5, 4, 7, 3, 6 ก็ได้คะแนนเต็มเช่นกัน ขึ้นกับลำดับการ Enqueue 3 และ 7 ในรอบที่ 4)

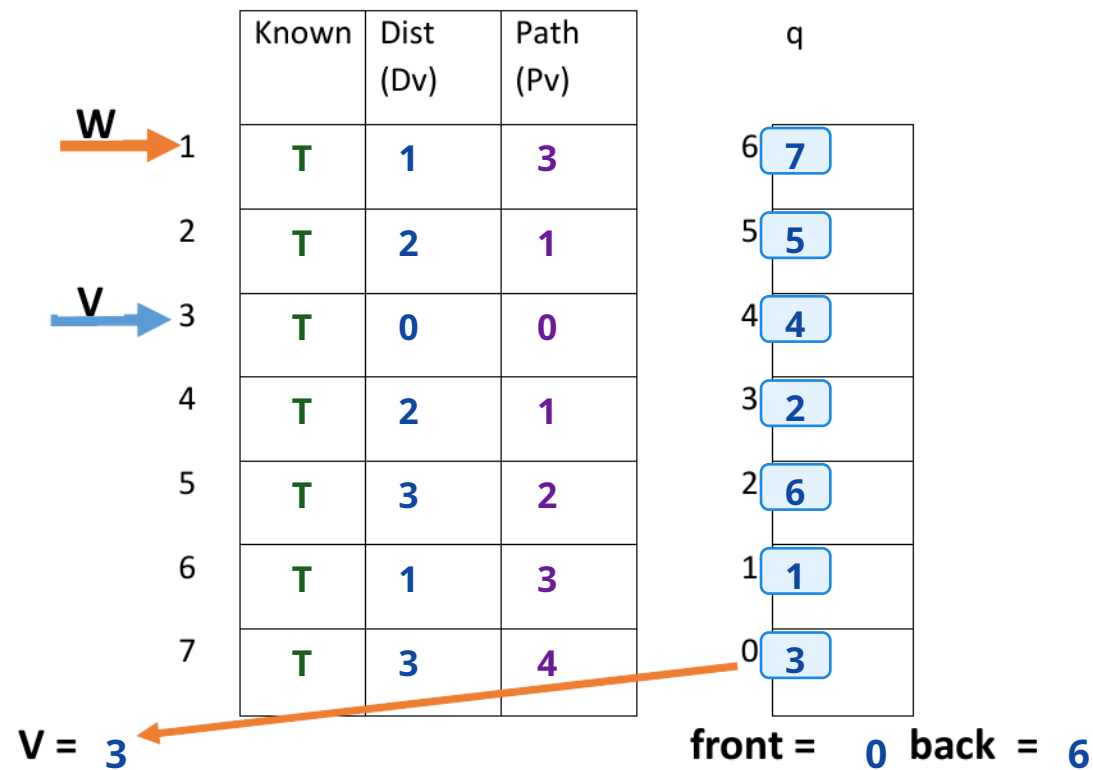
⚠ คำเตือนจุดได้ 0 คะแนน: ต้องคั่นด้วยเครื่องหมายจุลภาค (Comma) ห้ามเขียนลูกศร 1 -> 2 -> 5... เด็ดขาด!

Shortest Path

1. แปลงภาพกราฟ มาเป็น adjacency list ก่อน



2. สร้าง อะเรย์สำหรับการประมวลผล Shortest Path



```

if w.dist == INFINITY :
    w.dist = v.dist + 1
    w.path = v
    q.enqueue (w)
  
```

จงหาเส้นทางที่สั้นที่สุด จาก v3 ไป v5 จาก ตารางที่คำนวณได้ และ ความยาวของเส้นทางนั้น เท่ากับเท่าไร

After computing the shortest path, write down the length of the shortest path and the shortest path and from Vertex 3 to Vertex 5 from the calculated table.

ตอบ The length of the shortest path from Vertex 3 to Vertex 5 = **3**

The shortest path from Vertex 3 to Vertex 5 = **3, 1, 2, 5 (หรือ 3, 1, 4, 5)**

 **วิธีไล่สายย้อนกลับ (Backtracking from calculated table):**

- ดูที่โหนด 5 ในตารางหน้า 4: Dist(5) = 3 และ Path(5) = 2 (หรือ 4)
- ถอยกลับ: **5 <- 2 <- 1 <- 3 => ได้วิถีสั้นสุด: 3, 1, 2, 5 (ความยาว 3 เส้นเชื่อม)**

After computing the shortest path, write down the length of the shortest path and the shortest path and from Vertex 3 to Vertex 7 from the calculated table.

ตอบ The length of the shortest path from Vertex 3 to Vertex 7 = **3**

The shortest path from Vertex 3 to Vertex 7 = **3, 1, 4, 7**

 **วิธีไล่สายย้อนกลับ (Backtracking from calculated table):**

- ดูที่โหนด 7 ในตารางหน้า 4: Dist(7) = 3 และ Path(7) = 4
- ถอยกลับ: **7 <- 4 <- 1 <- 3 => ได้วิถีสั้นสุด: 3, 1, 4, 7 (ความยาว 3 เส้นเชื่อม)**

v4 ไป v5 **ความยาว = 1, วิถี = 4, 5** (มีเส้นเชื่อมตรงจาก 4 ไป 5 ใน Adjacency List)

✓ กฎเหล็กข้อสอบ: การเขียน Path ต้องคั่นด้วยเครื่องหมายจุลภาค (Comma) ห้ามเขียนลูกศร -> เด็ดขาด!